Day03 Pandas核心与数据清洗

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 3 天（8 课时）
> 主题：深入理解 Pandas 设计思想、DataFrame 操作、数据清洗实战

一、本日目标

1. 理解 Pandas 的设计思想：DataFrame 与 NumPy ndarray 的区别和联系；
2. 掌握数据读取与写入（CSV、Excel、不同编码）；
3. 掌握数据选择与过滤（loc/iloc/布尔索引/query）；
4. 掌握数据清洗（缺失值、重复值、异常值处理）；
5. 掌握数据转换与类型处理（astype、字符串操作、时间处理）；
6. 掌握分组聚合与透视表（groupby、agg、pivot_table）；
7. 掌握合并与连接（merge、concat、join）；
8. 项目实战：实现完整的数据清洗模块（9条清洗规则）。

二、知识要点

- Pandas 的核心是 DataFrame（二维表格）和 Series（一维序列），底层基于 NumPy；
- DataFrame 有行索引（index）和列索引（columns），支持异构数据类型；
- 数据清洗是本项目的第一步，9条规则全部基于 Pandas 实现；
- 关键坑：lac_id/cell_id 必须按字符串读取，time 必须按字符串读取，GBK 编码文件；
- 分组聚合是统计分析的核心，本项目按 imsi 分组处理每个用户。

课时1：Pandas设计思想与DataFrame

1.1 课时目标

理解 Pandas 的设计思想，掌握 DataFrame 和 Series 的创建与核心属性。

1.2 知识点深度讲解

1.2.1 Pandas 是什么

Pandas 是 Python 中最流行的数据分析库，名字来源于 "Panel Data"（面板数据）。它提供了两种核心数据结构：

- **Series**：一维带标签的数组，类似"带索引的 NumPy 数组"；
- **DataFrame**：二维带标签的表格，类似"Excel 表格"或"SQL 表"。

Pandas 底层基于 NumPy，但增加了：

1. **行索引和列索引**：可以用标签访问数据，不只是整数位置；
2. **异构数据类型**：不同列可以有不同类型（一列 int、一列 str、一列 float）；
3. **缺失值处理**：内置 NaN（Not a Number）表示缺失值；
4. **时间序列支持**：内置 DatetimeIndex，方便处理时间数据；
5. **分组聚合**：groupby 实现类似 SQL 的 GROUP BY；
6. **表格合并**：merge 实现类似 SQL 的 JOIN。

1.2.2 Series 创建与属性

```
import pandas as pd
import numpy as np

# 从列表创建（默认整数索引）
s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)
# 0    1
# 1    2
# 2    3
# 3    4
# 4    5
# dtype: int64

# 指定索引
s2 = pd.Series([1, 2, 3], index=["a", "b", "c"])
print(s2)
# a    1
# b    2
# c    3
# dtype: int64

# 从字典创建
s3 = pd.Series({"a": 1, "b": 2, "c": 3})

# 核心属性
print(s1.values)    # [1 2 3 4 5]（底层 NumPy 数组）
print(s1.index)     # RangeIndex(start=0, stop=5, step=1)
print(s1.dtype)     # int64
print(s1.shape)     # (5,)
print(s1.size)      # 5
```

1.2.3 DataFrame 创建与属性

```
import pandas as pd

# 方式1：从字典创建（最常用）
df = pd.DataFrame({
    "imsi": ["460000095007329090", "460000095007329091", "460000095007329092"],
    "time": ["20181003080000", "20181003080100", "20181003080200"],
    "lon": [123.41, 123.42, 123.43],
    "lat": [41.80, 41.81, 41.82]
})
print(df)
#                  imsi            time     lon    lat
# 0  460000095007329090  20181003080000  123.41  41.80
# 1  460000095007329091  20181003080100  123.42  41.81
# 2  460000095007329092  20181003080200  123.43  41.82

# 方式2：从列表的列表创建
df2 = pd.DataFrame(
    [[1, 2, 3], [4, 5, 6]],
    columns=["a", "b", "c"],
    index=["x", "y"]
)

# 核心属性
print(df.shape)       # (3, 4)（3行4列）
print(df.columns)     # Index(['imsi', 'time', 'lon', 'lat'], dtype='object')
print(df.index)       # RangeIndex(start=0, stop=3, step=1)
print(df.dtypes)      # 每列的数据类型
# imsi     object
# time     object
# lon     float64
# lat     float64
# dtype: object
print(df.values)      # 底层 NumPy 数组（object类型，因为有字符串）
print(df.info())      # 详细信息（每列非空数、类型、内存）
print(df.describe())  # 数值列的统计描述
```

1.2.4 DataFrame vs NumPy ndarray

特性	NumPy ndarray	Pandas DataFrame
维度	N维	2维（表格）
数据类型	全数组同类型	每列可不同类型
索引	整数位置	行标签+列标签
缺失值	无内置支持	NaN 内置支持
列操作	需手动索引	df["列名"] 直接访问
分组聚合	需手动实现	groupby 内置
表格合并	需手动实现	merge 内置
适用场景	数值计算、矩阵运算	表格数据、数据分析

> 本项目中，原始数据是表格（imsi、time、lac_id、cell_id、lon、lat），用 DataFrame 处理最方便。但距离计算、k-means 等数值计算用 NumPy 更快。两者配合使用：DataFrame 做数据清洗和管理，NumPy 做数值计算。

1.2.5 本项目中的 DataFrame 结构

清洗后的数据结构：

```
            imsi            time  lac_id  cell_id      lon      lat
0  460000095007329090  20181003080000   16789  67567924  123.41  41.80
1  460000095007329090  20181003080100   16789  67567925  123.42  41.81
...
```

- imsi：用户标识（字符串，15-20位数字）
- time：时间戳（字符串，YYYYMMDDHHMMSS 格式）
- lac_id：位置区码（字符串，不能按数字读！）
- cell_id：小区标识（字符串，不能按数字读！）
- lon：经度（float64）
- lat：纬度（float64）

1.3 项目实现步骤

1. 创建 Series 和 DataFrame；
2. 查看核心属性；
3. 理解 DataFrame 与 ndarray 的区别。

1.4 完整代码（DataFrame 创建与属性练习）

```
# ============================================================
# 课时1：Pandas设计思想与DataFrame
# ============================================================

import pandas as pd
import numpy as np

# ------------------------------------------------------------
# 1. Series 创建与属性
# ------------------------------------------------------------
print("=== 1. Series ===")
s1 = pd.Series([1, 2, 3, 4, 5])
print(f"默认索引:\n{s1}")
print(f"values: {s1.values}")
print(f"index: {s1.index}")
print(f"dtype: {s1.dtype}")

s2 = pd.Series([1, 2, 3], index=["a", "b", "c"])
print(f"\n指定索引:\n{s2}")
print(f"s2['a'] = {s2['a']}")

# ------------------------------------------------------------
# 2. DataFrame 创建
# ------------------------------------------------------------
print("\n=== 2. DataFrame 创建 ===")
df = pd.DataFrame({
    "imsi": ["460000095007329090", "460000095007329091", "460000095007329092"],
    "time": ["20181003080000", "20181003080100", "20181003080200"],
    "lon": [123.41, 123.42, 123.43],
    "lat": [41.80, 41.81, 41.82]
})
print(f"DataFrame:\n{df}")

# ------------------------------------------------------------
# 3. 核心属性
# ------------------------------------------------------------
```

```
print("\n=== 3. 核心属性 ===")
print(f"shape: {df.shape}（{df.shape[0]}行{df.shape[1]}列）")
print(f"columns: {df.columns.tolist()}")
print(f"index: {df.index}")
print(f"\ndtypes:\n{df.dtypes}")
print(f"\ninfo():")
df.info()

# ------------------------------------------------------------
# 4. describe 统计描述
# ------------------------------------------------------------
print("\n=== 4. describe 统计描述 ===")
print(df.describe())

# ------------------------------------------------------------
# 5. DataFrame vs ndarray
# ------------------------------------------------------------
print("\n=== 5. DataFrame vs ndarray ===")
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(f"ndarray shape: {arr.shape}, dtype: {arr.dtype}（全数组同类型）")
print(f"DataFrame 每列可不同类型: imsi={df['imsi'].dtype}, lon={df['lon'].dtype}")
print("→ DataFrame 适合表格数据，ndarray 适合数值计算")
```

1.5 运行结果与解读

运行后展示 Series、DataFrame 的创建和属性。特别注意：
- DataFrame 每列可以有不同类型（imsi 是 object，lon 是 float64）；
- `df.info()` 显示每列的非空数和类型，是数据探查的第一步；
- `df.describe()` 只显示数值列的统计描述。

1.6 当堂练习

1. 创建一个包含 5 个学生信息的 DataFrame（姓名、年龄、成绩），查看其 shape、dtypes、info；
2. 从字典创建一个 Series，索引为城市名，值为人口数，访问"北京"的人口；
3. 思考：为什么 DataFrame 的字符串列 dtype 显示为 object 而不是 str？（提示：Pandas 用 object 类型存储任意 Python 对象，包括字符串）

1.7 课时小结

- Pandas 核心是 Series（一维）和 DataFrame（二维表格）；
- DataFrame 有行索引和列索引，支持异构数据类型；
- DataFrame 底层基于 NumPy，但增加了标签、缺失值、分组、合并等功能；
- 本项目用 DataFrame 做数据清洗和管理，用 NumPy 做数值计算。

课时2：数据读取与写入

2.1 课时目标

掌握 Pandas 读取和写入 CSV/Excel 文件，处理不同编码、不同分隔符、指定列类型。

2.2 知识点深度讲解

2.2.1 读取 CSV 文件

`pd.read_csv()` 是最常用的读取函数，参数非常多，核心参数：

```
import pandas as pd

# 基本读取
df = pd.read_csv("data.csv")

# 指定编码（重要！）
df = pd.read_csv("data.csv", encoding="utf-8")    # UTF-8
df = pd.read_csv("data.csv", encoding="gbk")       # GBK（中文Windows默认）
df = pd.read_csv("data.csv", encoding="utf-8-sig") # UTF-8 with BOM

# 指定分隔符
df = pd.read_csv("data.csv", sep=",")    # 逗号（默认）
df = pd.read_csv("data.csv", sep="\t")   # 制表符（TSV）
df = pd.read_csv("data.csv", sep="|")    # 竖线

# 指定列名
df = pd.read_csv("data.csv", names=["col1", "col2", "col3"])  # 不读第一行作为列名
df = pd.read_csv("data.csv", header=0)   # 第一行作为列名（默认）

# 指定列类型（极其重要！本项目的坑）
df = pd.read_csv("data.csv", dtype={
    "lac_id": str,    # 按字符串读取，不转成数字
    "cell_id": str,
    "time": str,
    "imsi": str
})

# 只读取指定列
df = pd.read_csv("data.csv", usecols=["imsi", "time", "lon", "lat"])

# 读取前N行（大文件探查用）
df = pd.read_csv("data.csv", nrows=1000)

# 跳过前N行
df = pd.read_csv("data.csv", skiprows=2)
```

```
# 处理缺失值
df = pd.read_csv("data.csv", na_values=["", "NA", "null", "#N/A"])
```

> 本项目关键坑：`lac_id` 和 `cell_id` 必须用 `dtype=str` 按字符串读取！如果不指定，pandas 会推断为数字（int64 或 float64），导致 `16789` 变成 `16789.0`，与基站表的 `laci`（`16789-67567924`）拼接时对不上，关联后 0 行。
>
> 另一个坑：`time` 列也必须用 `dtype=str` 读取！如果不指定，pandas 会推断为 int64，`pd.Timestamp(time)` 会按纳秒解释，导致时间跨度只有微秒，速度计算异常。

2.2.2 读取 Excel 文件

```
# 读取 Excel
df = pd.read_excel("data.xlsx")
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")   # 指定工作表
df = pd.read_excel("data.xlsx", sheet_name=0)            # 按索引指定

# 读取多个工作表
xl = pd.ExcelFile("data.xlsx")
print(xl.sheet_names)   # 所有工作表名
df1 = xl.parse("Sheet1")
df2 = xl.parse("Sheet2")
```

2.2.3 写入 CSV 文件

```
# 基本写入
df.to_csv("output.csv")

# 不写索引
df.to_csv("output.csv", index=False)

# 指定编码（重要！）
df.to_csv("output.csv", encoding="utf-8")       # UTF-8
df.to_csv("output.csv", encoding="utf-8-sig")   # UTF-8 with BOM（Excel打开不乱码）
df.to_csv("output.csv", encoding="gbk")          # GBK

# 指定分隔符
df.to_csv("output.csv", sep="\t")

# 只写指定列
df.to_csv("output.csv", columns=["imsi", "time", "label"])

# 追加模式（不覆盖）
df.to_csv("output.csv", mode="a", header=False)
```

> 本项目中，输出文件用 `encoding="utf-8"`，但如果需要用 Excel 打开，建议用 `utf-8-sig`（带 BOM），否则 Excel 打开中文会乱码。

2.2.4 本项目的数据文件清单

文件	编码	行数	列数	用途
原始数据.csv	UTF-8	532828	10	信令原始数据
基站经纬度数据.csv	UTF-8	49954	3	基站→经纬度映射
商业区数据.csv	UTF-8	104	3	商圈中心经纬度
出行目的静态数据.csv	GBK	1273	4	居住/购物/旅游点
出行方式静态数据.csv	GBK	968	5	地铁/公交站点
实时流数据.csv	UTF-8	256212	3	实时人流数据
hive模块静态数据.csv	UTF-8	225	2	辅助数据

> 注意：出行目的和出行方式静态数据是 GBK 编码，读取时必须指定 `encoding="gbk"`，否则会报错或乱码。

2.3 项目实现步骤

1. 读取原始数据（指定 dtype 和 encoding）；
2. 读取基站数据（指定 dtype）；
3. 读取 GBK 编码的静态数据；
4. 写入清洗后的数据。

2.4 完整代码（数据读取与写入练习）

```
# ============================================================
# 课时2：数据读取与写入
# ============================================================

import pandas as pd
import os

# 数据根目录（根据实际路径修改）
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统(\09.交通时空大数据分析系统\B-项目数据"

# ------------------------------------------------------------
# 1. 读取原始数据（关键：指定 dtype）
# ------------------------------------------------------------
print("=== 1. 读取原始数据 ===")
raw_path = os.path.join(DATA_DIR, "原始数据.csv")

# 错误示范：不指定 dtype（lac_id/cell_id/time 会被推断为数字）
df_wrong = pd.read_csv(raw_path, nrows=5)
print("错误示范（不指定dtype）:")
print(df_wrong.dtypes)
print(f"lac_id 样例: {df_wrong['lac_id'].iloc[0]}（可能变成float！）")

# 正确示范：指定 dtype
df = pd.read_csv(raw_path, nrows=5, dtype={
    "imsi": str,
    "time": str,
    "lac_id": str,
    "cell_id": str
})
print("\n正确示范（指定dtype=str）:")
print(df.dtypes)
print(f"lac_id 样例: {df['lac_id'].iloc[0]}（保持字符串）")
print(f"time 样例: {df['time'].iloc[0]}（保持字符串）")

# ------------------------------------------------------------
```

```
# 2. 读取基站数据
# ------------------------------------------------------------
print("\n=== 2. 读取基站数据 ===")
bs_path = os.path.join(DATA_DIR, "基站经纬度数据.csv")
df_bs = pd.read_csv(bs_path, nrows=5, dtype={"laci": str})
print(df_bs)
print(f"laci 样例: {df_bs['laci'].iloc[0]}（形如 16789-67567924）")

# ------------------------------------------------------------
# 3. 读取 GBK 编码文件
# ------------------------------------------------------------
print("\n=== 3. 读取 GBK 编码文件 ===")
purpose_path = os.path.join(DATA_DIR, "出行目的静态数据.csv")
try:
    df_purpose = pd.read_csv(purpose_path, nrows=5)   # 不指定编码
except Exception as e:
    print(f"不指定编码报错: {e}")

df_purpose = pd.read_csv(purpose_path, nrows=5, encoding="gbk")
print(f"指定 encoding='gbk' 成功:")
print(df_purpose.head())

# ------------------------------------------------------------
# 4. 读取商业区数据
# ------------------------------------------------------------
print("\n=== 4. 读取商业区数据 ===")
biz_path = os.path.join(DATA_DIR, "商业区数据.csv")
df_biz = pd.read_csv(biz_path, nrows=10)
print(df_biz)

# ------------------------------------------------------------
# 5. 写入文件
# ------------------------------------------------------------
print("\n=== 5. 写入文件 ===")
output_dir = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
os.makedirs(output_dir, exist_ok=True)
```

```
# 写入测试
test_df = pd.DataFrame({
    "imsi": ["460000095007329090", "460000095007329091"],
    "time": ["20181003080000", "20181003080100"],
    "label": [0, 1]
})
test_path = os.path.join(output_dir, "test_output.csv")
test_df.to_csv(test_path, index=False, encoding="utf-8")
print(f"写入成功: {test_path}")

# 验证写入
df_read = pd.read_csv(test_path, dtype=str)
print(f"读回验证:\n{df_read}")

print("\n=== 关键总结 ===")
print("1. lac_id/cell_id/time 必须用 dtype=str 读取")
print("2. GBK 编码文件必须指定 encoding='gbk'")
print("3. 输出用 utf-8 或 utf-8-sig（Excel打开用后者）")
print("4. 大文件先用 nrows=1000 探查结构")
```

2.5 运行结果与解读

运行后展示：
- 不指定 dtype 时 lac_id 变成 float（`16789.0`），指定后保持字符串；
- GBK 文件不指定编码会报错，指定后正常读取；
- 写入和读回验证。

> 这三个坑（dtype、encoding、BOM）是本项目最容易踩的，必须在笔记中反复强调。

2.6 当堂练习

1. 用 `nrows=1000` 读取原始数据，查看每列的 dtype 和非空数；
2. 读取出行方式静态数据.csv（GBK编码），查看前5行；
3. 思考：为什么 `lac_id` 不能按数字读取？（提示：基站表的 laci 是 `lac_id-cell_id` 格式，需要字符串拼接）

2.7 课时小结

- `pd.read_csv()` 核心参数：encoding、sep、dtype、usecols、nrows；
- 关键坑：lac_id/cell_id/time 必须用 `dtype=str` 读取；
- GBK 编码文件必须指定 `encoding="gbk"`；
- 输出用 `utf-8` 或 `utf-8-sig`（Excel 打开用后者）；
- 大文件先用 `nrows=1000` 探查结构。

课时3：数据选择与过滤

3.1 课时目标

掌握 DataFrame 的数据选择和过滤方法：列选择、行选择、loc/iloc、布尔索引、query。

3.2 知识点深度讲解

3.2.1 列选择

```
import pandas as pd

df = pd.DataFrame({
    "imsi": ["A", "B", "C", "D", "E"],
    "time": ["0800", "0801", "0802", "0803", "0804"],
    "lon": [123.41, 123.42, 123.43, 123.44, 123.45],
    "lat": [41.80, 41.81, 41.82, 41.83, 41.84],
    "label": [0, 1, 0, 1, 0]
})

# 选择单列（返回 Series）
df["imsi"]          # 推荐
df.imsi             # 属性访问（不推荐，列名有空格或特殊字符时失效）

# 选择多列（返回 DataFrame）
df[["imsi", "time", "lon"]]   # 注意是双层方括号

# 按条件选择列
df.select_dtypes(include=["float64"])   # 只选数值列
df.select_dtypes(exclude=["object"])     # 排除字符串列
```

3.2.2 行选择：loc vs iloc

这是 Pandas 最容易混淆的概念之一：
- **loc**：按**标签**（label）选择，包含起始和结束；
- **iloc**：按**位置**（integer position）选择，不含结束（类似 Python 切片）。

```
# 用整数索引的 DataFrame
df = pd.DataFrame({"a": [1, 2, 3, 4, 5]}, index=[10, 20, 30, 40, 50])

# loc：按标签
df.loc[10]        # 索引标签为10的行（第0行）
df.loc[10:30]     # 标签10到30的行（包含30！3行）

# iloc：按位置
df.iloc[0]        # 第0行（标签10）
df.iloc[0:3]      # 第0到2行（不含3，3行）

# 同时选择行和列
df.loc[10:30, "a"]        # 标签10-30行，a列
df.iloc[0:3, 0]            # 第0-2行，第0列
df.iloc[0:3, [0, 1]]       # 第0-2行，第0和1列
```

> 记忆：loc = label（标签），iloc = integer location（整数位置）。loc 包含结束，iloc 不含结束。

3.2.3 布尔索引（最常用）

布尔索引用条件表达式选择行，是本项目最常用的过滤方式：

```
# 单条件
df[df["label"] == 1]           # 出行点
df[df["lon"] > 123.43]         # 经度大于123.43

# 多条件（& = 且，| = 或，~ = 非）
df[(df["label"] == 1) & (df["lon"] > 123.43)]   # 出行且经度>123.43
df[(df["label"] == 0) | (df["lat"] > 41.82)]     # 驻留或纬度>41.82
df[~(df["label"] == 1)]        # 非出行点（等价于 label==0）

# 用 isin 选择多个值
df[df["imsi"].isin(["A", "C", "E"])]   # imsi 为 A/C/E 的行

# 用 between 选择范围
df[df["lon"].between(123.42, 123.44)]   # 经度在123.42到123.44之间

# 用 str 方法过滤字符串列
df[df["imsi"].str.startswith("46000")]   # imsi 以46000开头
df[df["time"].str.contains("20181003")]   # time 包含20181003
```

> 注意：多条件必须用 `&` / `|` / `~`，不能用 `and` / `or` / `not`（后者是 Python 关键字，不支持向量化）。每个条件必须用括号包裹，否则优先级会出错。

3.2.4 query 方法

`query()` 方法用字符串表达式过滤，代码更简洁：

```
# 等价于 df[(df["label"] == 1) & (df["lon"] > 123.43)]
df.query("label == 1 and lon > 123.43")

# 支持变量
threshold = 123.43
df.query("label == 1 and lon > @threshold")   # @引用外部变量

# 支持 in
df.query("imsi in ['A', 'C', 'E']")
```

> query 方法在条件复杂时代码更可读，但性能略低于直接布尔索引。本项目用直接布尔索引即可。

3.2.5 本项目中的过滤场景

1. **过滤目标日期**：`df[df["time"].str.startswith("20181003")]`
2. **过滤出行点**：`df[df["label"] == 1]`
3. **过滤驻留点**：`df[df["label"] == 0]`
4. **过滤低频点**：`df[df["freq"] < 4]`
5. **按用户过滤**：`df[df["imsi"] == "460000095007329090"]`
6. **过滤有效经纬度**：`df[df["lon"].notna() & df["lat"].notna()]`

3.3 项目实现步骤

1. 练习列选择、行选择；
2. 练习 loc/iloc；
3. 练习布尔索引多条件过滤；
4. 模拟项目中的过滤场景。

3.4 完整代码（数据选择与过滤练习）

```
# ============================================================
# 课时3：数据选择与过滤
# ============================================================

import pandas as pd
import numpy as np

# 创建测试数据
df = pd.DataFrame({
    "imsi": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "time": ["201810030800", "201810030801", "201810030802", "201810030803",
             "201810030900", "201810030901", "201810040800", "201810040801"],
    "lon": [123.41, 123.42, 123.43, 123.44, 123.45, 123.46, 123.47, 123.48],
    "lat": [41.80, 41.81, 41.82, 41.83, 41.84, 41.85, 41.86, 41.87],
    "label": [0, 1, 0, 1, 0, 1, 0, 1],
    "freq": [5, 2, 8, 1, 3, 6, 4, 2]
})

print("=== 原始数据 ===")
print(df)

# ------------------------------------------------------------
# 1. 列选择
# ------------------------------------------------------------
print("\n=== 1. 列选择 ===")
print(f"单列（Series）:\n{df['imsi']}")
print(f"\n多列（DataFrame）:\n{df[['imsi', 'time', 'lon']]}")
print(f"\n数值列:\n{df.select_dtypes(include=['float64', 'int64'])}")

# ------------------------------------------------------------
# 2. loc vs iloc
# ------------------------------------------------------------
print("\n=== 2. loc vs iloc ===")
df2 = df.copy()
df2.index = [10, 20, 30, 40, 50, 60, 70, 80]   # 改索引为非连续整数
```

```
print(f"loc[10]（标签10）:\n{df2.loc[10]}")
print(f"\niloc[0]（位置0）:\n{df2.iloc[0]}")
print(f"\nloc[10:30]（标签10到30，包含30）:\n{df2.loc[10:30]}")
print(f"\niloc[0:3]（位置0到2，不含3）:\n{df2.iloc[0:3]}")

# ------------------------------------------------------------
# 3. 布尔索引
# ------------------------------------------------------------
print("\n=== 3. 布尔索引 ===")
print(f"出行点（label==1）:\n{df[df['label'] == 1]}")
print(f"\n出行且经度>123.44:\n{df[(df['label'] == 1) & (df['lon'] > 123.44)]}")
print(f"\n驻留或高频（freq>5）:\n{df[(df['label'] == 0) | (df['freq'] > 5)]}")
print(f"\nimsi 为 A/C/E:\n{df[df['imsi'].isin(['A', 'C', 'E'])]}")
print(f"\n经度在123.42到123.45:\n{df[df['lon'].between(123.42, 123.45)]}")

# ------------------------------------------------------------
# 4. 字符串过滤
# ------------------------------------------------------------
print("\n=== 4. 字符串过滤 ===")
print(f"20181003的数据:\n{df[df['time'].str.startswith('20181003')]}")
print(f"\n包含0900的数据:\n{df[df['time'].str.contains('0900')]}")

# ------------------------------------------------------------
# 5. query 方法
# ------------------------------------------------------------
print("\n=== 5. query 方法 ===")
print(f"出行且经度>123.44:\n{df.query('label == 1 and lon > 123.44')}")
threshold = 5
print(f"\n高频（freq>{threshold}）:\n{df.query('freq > @threshold')}")

# ------------------------------------------------------------
# 6. 项目模拟：过滤目标日期+有效用户
# ------------------------------------------------------------
print("\n=== 6. 项目模拟 ===")
# 过滤20181003的数据
df_target = df[df["time"].str.startswith("20181003")]
print(f"目标日期数据: {len(df_target)} 行")
```

```
# 过滤高频用户（freq>=4）
df_high_freq = df_target[df_target["freq"] >= 4]
print(f"高频用户数据：{len(df_high_freq)} 行")

# 过滤出行点
df_travel = df_high_freq[df_high_freq["label"] == 1]
print(f"出行点：{len(df_travel)} 行")
```

3.5 运行结果与解读

运行后展示各种选择和过滤方式。特别注意：
- loc 按标签，包含结束；iloc 按位置，不含结束；
- 多条件用 `&` / `|` / `~`，每个条件用括号；
- 字符串过滤用 `.str.startswith()` 、 `.str.contains()` 。

3.6 当堂练习

1. 用布尔索引从测试数据中选出"20181003的驻留点且经度>123.43"；
2. 用 `query` 方法实现同样的过滤；
3. 思考： `df.loc[0:5]` 和 `df.iloc[0:5]` 有什么区别？（提示：loc 包含索引5，iloc 不含位置5）

3.7 课时小结

- 列选择： `df["列名"]` 、 `df[["列1", "列2"]]` ；
- 行选择：loc 按标签（含结束），iloc 按位置（不含结束）；
- 布尔索引： `df[条件]` ，多条件用 `&` / `|` / `~` ，每个条件加括号；
- 字符串过滤： `.str.startswith()` 、 `.str.contains()` 、 `.str.endswith()` ；
- query 方法：字符串表达式，代码更简洁。

课时4：数据清洗（缺失值、重复值、异常值）

4.1 课时目标

掌握数据清洗的三大核心：缺失值处理、重复值处理、异常值处理。

4.2 知识点深度讲解

4.2.1 缺失值处理

Pandas 用 `NaN`（Not a Number）表示缺失值，检测和处理方法：

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "imsi": ["A", "B", "C", "D", "E"],
    "lon": [123.41, np.nan, 123.43, np.nan, 123.45],
    "lat": [41.80, 41.81, np.nan, 41.83, 41.84],
    "label": [0, 1, 0, np.nan, 1]
})

# 检测缺失值
df.isna()            # 返回布尔DataFrame，缺失为True
df.isnull()          # 等价于 isna()
df.notna()           # 非缺失为True
df.isna().sum()      # 每列缺失数
df.isna().mean()     # 每列缺失比例

# 删除缺失值
df.dropna()          # 删除任何含缺失值的行
df.dropna(axis=1)    # 删除任何含缺失值的列
df.dropna(subset=["lon", "lat"])   # 只看指定列，有缺失就删行
df.dropna(how="all") # 只删全为缺失的行
df.dropna(thresh=3)  # 至少3个非缺失值才保留

# 填充缺失值
df.fillna(0)                    # 用0填充
df.fillna({"lon": 0, "lat": 0}) # 不同列用不同值
df.fillna(method="ffill")       # 前向填充（用前一个非缺失值）
df.fillna(method="bfill")       # 后向填充（用后一个非缺失值）
df["lon"].fillna(df["lon"].mean())  # 用均值填充
df["lon"].fillna(df["lon"].median()) # 用中位数填充
df["lon"].interpolate()         # 线性插值填充
```

> 本项目中，原始数据的经纬度两列为空，需要通过基站关联补全。关联后仍为空的行（基站表中找不到）直接删除。

4.2.2 重复值处理

```
# 检测重复行
df.duplicated()            # 返回布尔Series，重复为True
df.duplicated().sum()      # 重复行数

# 检测指定列的重复
df.duplicated(subset=["imsi", "time"])   # imsi+time 重复

# 删除重复行
df.drop_duplicates()                        # 删除完全重复的行
df.drop_duplicates(subset=["imsi", "time"])  # 按指定列去重
df.drop_duplicates(keep="first")            # 保留第一个（默认）
df.drop_duplicates(keep="last")             # 保留最后一个
df.drop_duplicates(keep=False)              # 删除所有重复的
```

> 本项目中，清洗规则第8条：按 imsi+time 分钟去重（同一用户同一分钟只保留一条）。

4.2.3 异常值处理

异常值是指明显不合理的数据，检测方法：

```
# 方法1：统计方法（3σ原则）
mean = df["lon"].mean()
std = df["lon"].std()
lower = mean - 3 * std
upper = mean + 3 * std
df[(df["lon"] < lower) | (df["lon"] > upper)]   # 异常值

# 方法2：IQR方法（四分位距）
Q1 = df["lon"].quantile(0.25)
Q3 = df["lon"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df[(df["lon"] < lower) | (df["lon"] > upper)]   # 异常值

# 方法3：业务规则（本项目用这个）
# 经纬度范围：沈阳经度约123.0-124.0，纬度约41.5-42.5
df[(df["lon"] < 123.0) | (df["lon"] > 124.0)]   # 经度异常
df[(df["lat"] < 41.5) | (df["lat"] > 42.5)]      # 纬度异常

# 处理异常值
df = df[(df["lon"] >= 123.0) & (df["lon"] <= 124.0)]   # 删除异常
df["lon"] = df["lon"].clip(123.0, 124.0)                # 裁剪到范围
```

> 本项目中，异常值主要通过业务规则过滤：目标日期（2018-10-03）、有效经纬度范围、有效 imsi 格式。

4.2.4 本项目的9条清洗规则

1. **字段抽取**：从原始数据中提取需要的列（imsi、time、lac_id、cell_id）；
2. **去脏 imsi**：去除包含 `#`、`*`、`^` 等特殊字符的 imsi；
3. **去空 lac_id**：去除 lac_id 或 cell_id 为空的行；
4. **时间格式转换**：毫秒时间戳转 YYYYMMDDHHMMSS 格式；
5. **过滤目标日期**：只保留 2018-10-03 的数据；
6. **基站关联补经纬度**：通过 lac_id+cell_id 关联基站表，补全经纬度；
7. **按 imsi+time 排序**：每个用户按时间排序；

8. **分钟去重**：同一用户同一分钟只保留一条；
9. **过滤低频用户**：单用户数据量 < 20 条的用户删除。

> 这9条规则是本项目数据清洗的核心，每一条都有明确的业务理由，课时8会完整实现。

4.3 项目实现步骤

1. 练习缺失值检测和处理；
2. 练习重复值检测和去重；
3. 练习异常值检测和处理；
4. 理解本项目的9条清洗规则。

4.4 完整代码（数据清洗练习）

```
# ============================================================
# 课时4：数据清洗（缺失值、重复值、异常值）
# ============================================================

import pandas as pd
import numpy as np

# 创建含缺失值、重复值、异常值的测试数据
df = pd.DataFrame({
    "imsi": ["A", "A", "B", "C", "C", "D", "E", "F"],
    "time": ["0800", "0800", "0801", "0802", "0802", "0803", "0804", "0805"],
    "lon": [123.41, 123.41, np.nan, 123.43, 123.43, 999.0, 123.45, 123.46],
    "lat": [41.80, 41.80, 41.81, np.nan, np.nan, 41.83, 41.84, 41.85],
    "label": [0, 0, 1, 0, 0, 1, 0, 1]
})

print("=== 原始数据（含缺失、重复、异常）===")
print(df)

# ------------------------------------------------------------
# 1. 缺失值处理
# ------------------------------------------------------------
print("\n=== 1. 缺失值处理 ===")
print(f"每列缺失数:\n{df.isna().sum()}")
print(f"\n每列缺失比例:\n{df.isna().mean().round(3)}")

# 删除经纬度缺失的行
df_clean = df.dropna(subset=["lon", "lat"])
print(f"\n删除经纬度缺失后：{len(df_clean)} 行（原{len(df)}行）")

# 用均值填充（演示，本项目用删除）
df_fill = df.copy()
df_fill["lon"] = df_fill["lon"].fillna(df_fill["lon"].mean())
df_fill["lat"] = df_fill["lat"].fillna(df_fill["lat"].median())
print(f"\n填充后缺失数：lon={df_fill['lon'].isna().sum()}, lat=
{df_fill['lat'].isna().sum()}")
```

```
# ------------------------------------------------------------
# 2. 重复值处理
# ------------------------------------------------------------
print("\n=== 2. 重复值处理 ===")
print(f"完全重复行数: {df.duplicated().sum()}")
print(f"imsi+time 重复行数: {df.duplicated(subset=['imsi', 'time']).sum()}")

# 按 imsi+time 去重（保留第一条）
df_dedup = df.drop_duplicates(subset=["imsi", "time"], keep="first")
print(f"去重后: {len(df_dedup)} 行（原{len(df)}行）")

# ------------------------------------------------------------
# 3. 异常值处理
# ------------------------------------------------------------
print("\n=== 3. 异常值处理 ===")
# 业务规则：沈阳经度 123.0-124.0，纬度 41.5-42.5
lon_outliers = df[(df["lon"] < 123.0) | (df["lon"] > 124.0)]
print(f"经度异常行:\n{lon_outliers}")

# 删除异常值
df_normal = df[(df["lon"] >= 123.0) & (df["lon"] <= 124.0) &
               (df["lat"] >= 41.5) & (df["lat"] <= 42.5)]
print(f"\n删除异常后: {len(df_normal)} 行（原{len(df)}行）")

# 3σ方法（演示）
print("\n3σ方法检测:")
for col in ["lon", "lat"]:
    if df_normal[col].notna().sum() > 0:
        mean = df_normal[col].mean()
        std = df_normal[col].std()
        print(f"  {col}: mean={mean:.2f}, std={std:.2f}, 范围=[{mean-3*std:.2f},
{mean+3*std:.2f}]")

# ------------------------------------------------------------
# 4. 综合清洗流程
# ------------------------------------------------------------
print("\n=== 4. 综合清洗流程 ===")
```

```
print(f"原始数据：{len(df)} 行")

# 步骤1：删除经纬度缺失
step1 = df.dropna(subset=["lon", "lat"])
print(f"步骤1（删缺失）：{len(step1)} 行")

# 步骤2：删除异常经纬度
step2 = step1[(step1["lon"] >= 123.0) & (step1["lon"] <= 124.0) &
              (step1["lat"] >= 41.5) & (step1["lat"] <= 42.5)]
print(f"步骤2（删异常）：{len(step2)} 行")

# 步骤3：按 imsi+time 去重
step3 = step2.drop_duplicates(subset=["imsi", "time"], keep="first")
print(f"步骤3（去重）：{len(step3)} 行")

# 步骤4：按 imsi+time 排序
step4 = step3.sort_values(by=["imsi", "time"]).reset_index(drop=True)
print(f"步骤4（排序）：{len(step4)} 行")
print(f"\n清洗后数据:\n{step4}")

print("\n=== 本项目9条清洗规则 ===")
print("1. 字段抽取")
print("2. 去脏imsi（含#*^等特殊字符）")
print("3. 去空lac_id/cell_id")
print("4. 时间格式转换（毫秒→YYYYMMDDHHMMSS）")
print("5. 过滤目标日期（2018-10-03）")
print("6. 基站关联补经纬度")
print("7. 按imsi+time排序")
print("8. 分钟去重")
print("9. 过滤低频用户（<20条）")
```

4.5 运行结果与解读

运行后展示缺失值、重复值、异常值的检测和处理，以及综合清洗流程。特别注意：
- 本项目用业务规则（经纬度范围、目标日期）检测异常值，不用统计方法；
- 清洗流程是顺序的，每一步都可能减少数据量；
- 9条清洗规则是本项目的核心，课时8会完整实现。

4.6 当堂练习

1. 用 3σ 方法检测测试数据中 lon 列的异常值；
2. 用 IQR 方法检测同样的异常值，对比结果差异；
3. 思考：为什么本项目用业务规则检测异常值，而不用统计方法？（提示：经纬度有明确的地理范围，统计方法可能误删边界点）

4.7 课时小结

- 缺失值：`isna()` 检测，`dropna()` 删除，`fillna()` 填充；
- 重复值：`duplicated()` 检测，`drop_duplicates()` 去重；
- 异常值：统计方法（3σ、IQR）或业务规则，本项目用业务规则；
- 本项目9条清洗规则：字段抽取、去脏imsi、去空、时间转换、日期过滤、基站关联、排序、去重、过滤低频。

课时5：数据转换与类型处理

5.1 课时目标

掌握数据类型转换、字符串操作、时间处理、列操作。

5.2 知识点深度讲解

5.2.1 数据类型转换

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "imsi": ["460000095007329090", "460000095007329091"],
    "time": ["20181003080000", "20181003080100"],
    "lon": ["123.41", "123.42"],   # 字符串形式的数字
    "lat": ["41.80", "41.81"],
    "label": ["0", "1"]
})

# 查看类型
print(df.dtypes)
# imsi     object
# time     object
# lon      object
# lat      object
# label    object

# 转换类型
df["lon"] = df["lon"].astype(float)      # 字符串→浮点
df["lat"] = df["lat"].astype(float)
df["label"] = df["label"].astype(int)     # 字符串→整数
df["label"] = df["label"].astype("int8")  # 整数→小整数（省内存）

# 安全转换（errors="coerce"：无法转换的变NaN）
df["lon"] = pd.to_numeric(df["lon"], errors="coerce")
df["time"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S", errors="coerce")

# 转换为分类类型（省内存，适合有限取值的列）
df["label"] = df["label"].astype("category")
```

> 本项目中，`lac_id` 和 `cell_id` 必须保持字符串类型，不能转成数字。`time` 也保持字符串，

> 因为需要用 `.str` 方法提取日期。经纬度从基站表关联后是 float64。

5.2.2 字符串操作（.str 访问器）

Pandas 的 Series 有 `.str` 访问器，提供向量化的字符串操作：

```
s = pd.Series(["Hello World", "Python Pandas", "Data Analysis"])

# 基本操作
s.str.lower()           # 转小写
s.str.upper()           # 转大写
s.str.len()             # 长度
s.str.strip()           # 去首尾空白
s.str.replace(" ", "_") # 替换

# 拆分
s.str.split(" ")        # 拆分成列表
# 0    [Hello, World]
# 1    [Python, Pandas]
# 2    [Data, Analysis]

s.str.split(" ", expand=True)   # 拆分成多列
#         0         1
# 0   Hello     World
# 1  Python    Pandas
# 2    Data  Analysis

# 提取
s.str.extract(r"(\w+)")  # 正则提取第一个单词

# 判断
s.str.contains("Python")  # 是否包含
s.str.startswith("Hello") # 是否以...开头
s.str.endswith("sis")     # 是否以...结尾

# 切片
s.str[0:5]            # 前5个字符
s.str[-3:]             # 后3个字符
```

本项目中，`.str` 操作大量使用：
- `time.str[:8]` 提取日期（YYYYMMDD）
- `time.str[8:10]` 提取小时
- `imsi.str.contains("[#* ^]")` 检测脏 imsi
- `lac_id + "-" + cell_id` 拼接 laci

5.2.3 时间处理

```
# 字符串转时间
df["time"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S")

# 时间属性
df["time"].dt.year        # 年
df["time"].dt.month       # 月
df["time"].dt.day         # 日
df["time"].dt.hour        # 小时
df["time"].dt.minute      # 分钟
df["time"].dt.second      # 秒
df["time"].dt.dayofweek   # 星期几（0=周一）
df["time"].dt.date        # 日期部分

# 时间差
df["time_diff"] = df["time"].diff()   # 相邻行时间差
df["time_diff"].dt.total_seconds()     # 时间差转秒

# 时间格式化
df["time"].dt.strftime("%Y-%m-%d %H:%M:%S")   # 转字符串

# 时间过滤
df[df["time"] >= "2018-10-03 08:00:00"]
df[df["time"].between("2018-10-03 08:00", "2018-10-03 09:00")]
```

注意：pandas 3.0 中 `Series.strftime` 已废弃，改用 `.dt.strftime`。这是本项目的一个坑。

本项目中，time 列保持字符串格式（YYYYMMDDHHMMSS），不转成 datetime，因为：
1. 字符串排序和 datetime 排序结果一致（固定长度）；
2. 字符串提取日期/小时用 `.str` 更方便；

> 3. 转 datetime 后再格式化会有性能开销。
> 但计算时间差时，需要转成 datetime 或手动解析。

5.2.4 列操作

```
# 新增列
df["new_col"] = 0                    # 常量列
df["lon_lat"] = df["lon"].astype(str) + "," + df["lat"].astype(str)  # 计算列
df["hour"] = df["time"].str[8:10]    # 从其他列提取

# 删除列
df = df.drop("new_col", axis=1)       # 删除单列
df = df.drop(["col1", "col2"], axis=1) # 删除多列
del df["new_col"]                      # 另一种方式

# 重命名列
df = df.rename(columns={"old_name": "new_name"})
df.columns = ["new_col1", "new_col2", ...]  # 全部重命名

# 列顺序调整
df = df[["imsi", "time", "lon", "lat", "label"]]  # 按指定顺序
```

5.2.5 apply 与 map

```
# apply：对每行/每列应用函数
df["full_name"] = df.apply(lambda row: f"{row['imsi']}_{row['time']}", axis=1)

# map：对 Series 的每个值应用函数或映射
df["label_name"] = df["label"].map({0: "驻留", 1: "出行"})

# applymap：对 DataFrame 的每个元素应用函数（已废弃，改用 map）
df = df.map(lambda x: x.strip() if isinstance(x, str) else x)
```

> 注意：apply 本质上还是 Python 循环，性能不如向量化。只有在逻辑复杂无法向量化时才用 apply。

5.3 项目实现步骤

1. 练习类型转换；
2. 练习字符串操作；
3. 练习时间处理；
4. 练习列操作。

5.4 完整代码（数据转换练习）

```
# ============================================================
# 课时5：数据转换与类型处理
# ============================================================

import pandas as pd
import numpy as np

# 创建测试数据
df = pd.DataFrame({
    "imsi": ["460000095007329090", "460000095007329091", "460000095007329092"],
    "time": ["20181003080000", "20181003090100", "20181003100200"],
    "lon": ["123.41", "123.42", "123.43"],
    "lat": ["41.80", "41.81", "41.82"],
    "label": ["0", "1", "0"]
})

print("=== 原始数据 ===")
print(df)
print(f"\ndtypes:\n{df.dtypes}")

# ------------------------------------------------------------
# 1. 类型转换
# ------------------------------------------------------------
print("\n=== 1. 类型转换 ===")
df["lon"] = df["lon"].astype(float)
df["lat"] = df["lat"].astype(float)
df["label"] = df["label"].astype("int8")
print(f"转换后 dtypes:\n{df.dtypes}")
print(f"label 内存: {df['label'].nbytes} 字节（int8 vs int64=
{df['label'].astype('int64').nbytes}）")

# ------------------------------------------------------------
# 2. 字符串操作
# ------------------------------------------------------------
print("\n=== 2. 字符串操作 ===")
print(f"time 前8位（日期）: {df['time'].str[:8].tolist()}")
```

```
print(f"time 第9-10位（小时）: {df['time'].str[8:10].tolist()}")
print(f"imsi 长度: {df['imsi'].str.len().tolist()}")
print(f"imsi 是否以460开头: {df['imsi'].str.startswith('460').tolist()}")

# 拼接 laci（模拟基站关联）
df["lac_id"] = ["16789", "16789", "16790"]
df["cell_id"] = ["67567924", "67567925", "67567926"]
df["laci"] = df["lac_id"] + "-" + df["cell_id"]
print(f"\n拼接 laci: {df['laci'].tolist()}")

# 检测脏 imsi
dirty = pd.Series(["460000095007329090", "460000095007329#91", "460000095007329*92"])
print(f"\n脏 imsi 检测（含#*^）: {dirty.str.contains('[#* ^]').tolist()}")

# ------------------------------------------------------------
# 3. 时间处理
# ------------------------------------------------------------
print("\n=== 3. 时间处理 ===")
# 字符串转 datetime
df["time_dt"] = pd.to_datetime(df["time"], format="%Y%m%d%H%M%S")
print(f"time_dt:\n{df['time_dt']}")
print(f"小时: {df['time_dt'].dt.hour.tolist()}")
print(f"星期: {df['time_dt'].dt.dayofweek.tolist()}（0=周一）")

# 时间差
df["time_diff"] = df["time_dt"].diff()
print(f"\n时间差:\n{df['time_diff']}")
print(f"时间差（秒）: {df['time_diff'].dt.total_seconds().tolist()}")

# 格式化输出
print(f"\n格式化: {df['time_dt'].dt.strftime('%Y-%m-%d %H:%M:%S').tolist()}")

# ------------------------------------------------------------
# 4. 列操作
# ------------------------------------------------------------
print("\n=== 4. 列操作 ===")
# 新增列
df["hour"] = df["time"].str[8:10].astype(int)
```

```
df["date"] = df["time"].str[:8]
print(f"新增 hour, date 列:\n{df[['time', 'hour', 'date']]}")

# map 映射
df["label_name"] = df["label"].map({0: "驻留", 1: "出行"})
print(f"\nlabel 映射: {df['label_name'].tolist()}")

# 删除列
df = df.drop(["time_dt", "time_diff"], axis=1)
print(f"\n删除后列: {df.columns.tolist()}")

# ------------------------------------------------------------
# 5. 项目模拟：从 time 提取日期和小时
# ------------------------------------------------------------
print("\n=== 5. 项目模拟 ===")
# 模拟原始数据
raw = pd.DataFrame({
    "imsi": ["A"] * 5 + ["B"] * 5,
    "time": ["20181003080000", "20181003080100", "20181003080200",
             "20181003090000", "20181003090100",
             "20181003080000", "20181003083000", "20181003090000",
             "20181003100000", "20181003110000"]
})

# 提取日期和小时
raw["date"] = raw["time"].str[:8]
raw["hour"] = raw["time"].str[8:10].astype(int)
raw["minute"] = raw["time"].str[10:12].astype(int)

print(f"提取后:\n{raw}")
print(f"\n各小时数据量:\n{raw['hour'].value_counts().sort_index()}")
```

5.5 运行结果与解读

运行后展示类型转换、字符串操作、时间处理、列操作。特别注意：
- `.str` 操作是向量化的，比循环快；
- 时间处理用 `.dt` 访问器，pandas 3.0 用 `.dt.strftime` 而非 `.strftime`；
- 本项目 time 保持字符串，提取日期/小时用 `.str` 切片。

5.6 当堂练习

1. 从 time 列（YYYYMMDDHHMMSS）中提取年、月、日、时、分、秒，新增6列；
2. 用 `map` 将 label 列的 0/1 映射为"驻留"/"出行"；
3. 思考：为什么本项目 time 列保持字符串而不转 datetime？（提示：字符串排序和 datetime 一致，提取用 .str 更方便，性能更好）

5.7 课时小结

- 类型转换：`astype()`、`pd.to_numeric()`、`pd.to_datetime()`；
- 字符串操作：`.str` 访问器，lower/upper/split/extract/contains/startswith；
- 时间处理：`.dt` 访问器，year/month/day/hour/diff/total_seconds/strftime；
- 列操作：新增、删除、重命名、调整顺序；
- apply/map：复杂逻辑用，性能不如向量化。

课时6：分组聚合与透视表

6.1 课时目标

掌握 groupby 分组聚合、agg 多指标聚合、pivot_table 透视表。

6.2 知识点深度讲解

6.2.1 groupby 基本用法

`groupby` 是 Pandas 最强大的功能之一，实现类似 SQL 的 GROUP BY：

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "imsi": ["A", "A", "A", "B", "B", "B", "C", "C"],
    "hour": [8, 9, 10, 8, 9, 10, 8, 9],
    "lon": [123.41, 123.42, 123.43, 123.44, 123.45, 123.46, 123.47, 123.48],
    "lat": [41.80, 41.81, 41.82, 41.83, 41.84, 41.85, 41.86, 41.87],
    "label": [0, 1, 0, 1, 0, 1, 0, 1]
})

# 按 imsi 分组
grouped = df.groupby("imsi")

# 对分组应用聚合函数
grouped.mean()       # 每组均值
grouped.sum()        # 每组求和
grouped.count()      # 每组计数
grouped.max()        # 每组最大值
grouped.min()        # 每组最小值
grouped.std()        # 每组标准差

# 对指定列聚合
grouped["lon"].mean()    # 每组经度均值
grouped[["lon", "lat"]].mean()   # 每组经纬度均值
```

6.2.2 agg 多指标聚合

```
# 单指标
grouped["lon"].agg("mean")

# 多指标
grouped["lon"].agg(["mean", "std", "min", "max", "count"])

# 不同列不同指标
grouped.agg({
    "lon": ["mean", "std"],
    "lat": ["mean", "min", "max"],
    "label": "sum"
})

# 自定义聚合函数
grouped["lon"].agg(lambda x: x.max() - x.min())   # 极差
```

6.2.3 分组后过滤（filter）

```
# 过滤数据量 >= 3 的用户
df.groupby("imsi").filter(lambda x: len(x) >= 3)

# 过滤平均经度 > 123.43 的用户
df.groupby("imsi").filter(lambda x: x["lon"].mean() > 123.43)
```

> 本项目中，清洗规则第 9 条：过滤单用户数据量 < 20 条的用户，就是用 `groupby("imsi").filter(lambda x: len(x) >= 20)`。

6.2.4 分组后转换（transform）

`transform` 返回与原 DataFrame 同长度的结果，常用于"组内标准化"：

```
# 每个用户的经度减去该用户的平均经度
df["lon_centered"] = df.groupby("imsi")["lon"].transform(lambda x: x - x.mean())

# 每个用户的数据量
df["user_count"] = df.groupby("imsi")["imsi"].transform("count")
```

6.2.5 多列分组

```
# 按 imsi 和 hour 分组
df.groupby(["imsi", "hour"]).mean()

# 按多列分组后聚合
df.groupby(["imsi", "label"]).agg({
    "lon": "mean",
    "lat": ["mean", "count"]
})
```

6.2.6 透视表（pivot_table）

透视表是 groupby 的可视化形式，类似 Excel 的数据透视表：

```
# 基本透视表
pd.pivot_table(df, values="lon", index="imsi", columns="label", aggfunc="mean")

# 多指标
pd.pivot_table(df, values=["lon", "lat"], index="imsi", columns="label",
               aggfunc=["mean", "count"])

# 添加合计
pd.pivot_table(df, values="lon", index="imsi", columns="label",
               aggfunc="mean", margins=True, margins_name="合计")
```

6.2.7 本项目中的分组聚合场景

1. **按用户统计数据量**：`df.groupby("imsi").size()`
2. **过滤低频用户**：`df.groupby("imsi").filter(lambda x: len(x) >= 20)`
3. **按用户排序**：`df.sort_values(["imsi", "time"])`
4. **阶段1连续段识别**：按用户分组后识别连续段

5. **阶段3按用户聚类**：每个用户独立做 k-means
6. **出行目的统计**：按用户+目的类型分组
7. **出行方式统计**：按用户+方式类型分组
8. **实时热力**：按时间窗口+商圈分组统计人次

6.3 项目实现步骤

1. 练习 groupby 基本聚合；
2. 练习 agg 多指标聚合；
3. 练习 filter 过滤分组；
4. 练习透视表。

6.4 完整代码（分组聚合练习）

```
# ============================================================
# 课时6：分组聚合与透视表
# ============================================================

import pandas as pd
import numpy as np

# 创建测试数据
df = pd.DataFrame({
    "imsi": ["A", "A", "A", "A", "B", "B", "B", "C", "C"],
    "time": ["0800", "0801", "0900", "0901", "0800", "0830", "0900", "0800", "0900"],
    "hour": [8, 8, 9, 9, 8, 8, 9, 8, 9],
    "lon": [123.41, 123.42, 123.43, 123.44, 123.45, 123.46, 123.47, 123.48, 123.49],
    "lat": [41.80, 41.81, 41.82, 41.83, 41.84, 41.85, 41.86, 41.87, 41.88],
    "label": [0, 1, 0, 1, 0, 1, 0, 1, 0]
})

print("=== 原始数据 ===")
print(df)

# ------------------------------------------------------------
# 1. groupby 基本聚合
# ------------------------------------------------------------
print("\n=== 1. groupby 基本聚合 ===")
grouped = df.groupby("imsi")
print(f"每组数据量:\n{grouped.size()}")
print(f"\n每组均值:\n{grouped[['lon', 'lat']].mean()}")
print(f"\n每组经度统计:\n{grouped['lon'].agg(['mean', 'std', 'min', 'max', 'count'])}")

# ------------------------------------------------------------
# 2. agg 多指标多列
# ------------------------------------------------------------
print("\n=== 2. agg 多指标多列 ===")
result = grouped.agg({
    "lon": ["mean", "std"],
    "lat": ["mean", "min", "max"],
```

```
    "label": "sum"
})
print(result)

# 自定义聚合：极差
print(f"\n每组经度极差:\n{grouped['lon'].agg(lambda x: x.max() - x.min())}")

# ------------------------------------------------------------
# 3. filter 过滤分组
# ------------------------------------------------------------
print("\n=== 3. filter 过滤分组 ===")
# 过滤数据量 >= 3 的用户
df_filtered = df.groupby("imsi").filter(lambda x: len(x) >= 3)
print(f"数据量>=3的用户: {df_filtered['imsi'].unique()}")
print(f"过滤后: {len(df_filtered)} 行（原{len(df)}行）")

# 过滤平均经度 > 123.44 的用户
df_high_lon = df.groupby("imsi").filter(lambda x: x["lon"].mean() > 123.44)
print(f"\n平均经度>123.44的用户: {df_high_lon['imsi'].unique()}")

# ------------------------------------------------------------
# 4. transform 组内转换
# ------------------------------------------------------------
print("\n=== 4. transform 组内转换 ===")
# 每个用户的经度减去该用户均值
df["lon_centered"] = df.groupby("imsi")["lon"].transform(lambda x: x - x.mean())
print(f"组内中心化经度:\n{df[['imsi', 'lon', 'lon_centered']]}")

# 每个用户的数据量
df["user_count"] = df.groupby("imsi")["imsi"].transform("count")
print(f"\n用户数据量列:\n{df[['imsi', 'user_count']].drop_duplicates()}")

# ------------------------------------------------------------
# 5. 多列分组
# ------------------------------------------------------------
print("\n=== 5. 多列分组 ===")
# 按 imsi + label 分组
multi_group = df.groupby(["imsi", "label"]).agg({
```

```
    "lon": "mean",
    "lat": "mean"
})
print(multi_group)

# ------------------------------------------------------------
# 6. 透视表
# ------------------------------------------------------------
print("\n=== 6. 透视表 ===")
pivot = pd.pivot_table(df, values="lon", index="imsi", columns="label",
                       aggfunc="mean", margins=True, margins_name="合计")
print(pivot)

# ------------------------------------------------------------
# 7. 项目模拟：过滤低频用户
# ------------------------------------------------------------
print("\n=== 7. 项目模拟：过滤低频用户 ===")
# 模拟更多用户
np.random.seed(42)
users = [f"user_{i}" for i in range(20)]
imsi_list = []
for u in users:
    count = np.random.randint(5, 50)   # 每个用户5-50条
    imsi_list.extend([u] * count)
df_large = pd.DataFrame({
    "imsi": imsi_list,
    "time": [f"20181003{h:02d}0000" for h in np.random.randint(0, 24, len(imsi_list))],
    "lon": np.random.uniform(123.3, 123.5, len(imsi_list)),
    "lat": np.random.uniform(41.7, 41.9, len(imsi_list))
})

print(f"原始数据: {len(df_large)} 行, {df_large['imsi'].nunique()} 用户")

# 过滤数据量 < 20 条的用户
MIN_RECORDS = 20
user_counts = df_large.groupby("imsi").size()
valid_users = user_counts[user_counts >= MIN_RECORDS].index
df_valid = df_large[df_large["imsi"].isin(valid_users)]
```

```
print(f"过滤后：{len(df_valid)} 行，{df_valid['imsi'].nunique()} 用户（>= {MIN_RECORDS} 条）")

# 用 filter 实现（等价）
df_valid2 = df_large.groupby("imsi").filter(lambda x: len(x) >= MIN_RECORDS)
print(f"filter 方式：{len(df_valid2)} 行（结果一致：{len(df_valid) == len(df_valid2)}）")
```

6.5 运行结果与解读

运行后展示 groupby、agg、filter、transform、透视表。特别注意：
- `groupby("imsi").filter(lambda x: len(x) >= 20)` 是过滤低频用户的标准写法；
- `transform` 返回与原数据同长度，适合组内标准化；
- 透视表是 groupby 的可视化形式。

6.6 当堂练习

1. 用 groupby 统计每个用户的出行点数（label=1）和驻留点（label=0）；
2. 用透视表展示每个用户每小时的平均经度；
3. 思考：`filter` 和 `transform` 有什么区别？（提示：filter 返回过滤后的行，transform 返回与原数据同长度的转换结果）

6.7 课时小结

- groupby：按列分组，支持单指标和多指标聚合；
- agg：多指标多列聚合，支持自定义函数；
- filter：过滤分组（如低频用户）；
- transform：组内转换（如组内标准化）；
- 多列分组：`groupby(["col1", "col2"])`；
- pivot_table：透视表，类似 Excel 数据透视表；
- 本项目核心：过滤低频用户用 `groupby("imsi").filter(lambda x: len(x) >= 20)`。

课时7：合并与连接

7.1 课时目标

掌握 merge、concat、join 三种合并方式，理解不同连接类型。

7.2 知识点深度讲解

7.2.1 merge（类似 SQL JOIN）

`merge` 是最常用的合并方式，类似 SQL 的 JOIN：

```
import pandas as pd

# 左表：用户数据
df_left = pd.DataFrame({
    "imsi": ["A", "B", "C", "D"],
    "time": ["0800", "0801", "0802", "0803"],
    "lac_id": ["16789", "16789", "16790", "16791"],
    "cell_id": ["67567924", "67567925", "67567926", "67567927"]
})

# 右表：基站经纬度
df_right = pd.DataFrame({
    "laci": ["16789-67567924", "16789-67567925", "16790-67567926", "16792-67567928"],
    "lon": [123.41, 123.42, 123.43, 123.44],
    "lat": [41.80, 41.81, 41.82, 41.83]
})

# 先在左表拼接 laci
df_left["laci"] = df_left["lac_id"] + "-" + df_left["cell_id"]

# 内连接（只保留匹配的）
df_inner = pd.merge(df_left, df_right, on="laci", how="inner")

# 左连接（保留左表所有行，未匹配的为NaN）
df_left_join = pd.merge(df_left, df_right, on="laci", how="left")

# 右连接（保留右表所有行）
df_right_join = pd.merge(df_left, df_right, on="laci", how="right")

# 外连接（保留两边所有行）
df_outer = pd.merge(df_left, df_right, on="laci", how="outer")
```

连接类型对比：

类型	保留	类似 SQL
inner	两边都匹配的	INNER JOIN
left	左表所有，右表匹配的	LEFT JOIN
right	右表所有，左表匹配的	RIGHT JOIN
outer	两边所有	FULL OUTER JOIN

> 本项目中，基站关联补经纬度用左连接（`how="left"`），保留所有原始数据，未匹配到基站的经纬度为 NaN，后续删除。

7.2.2 不同列名的 merge

```
# 左右表列名不同时，用 left_on 和 right_on
pd.merge(df_left, df_right, left_on="laci", right_on="laci", how="left")

# 合并后删除重复列
pd.merge(df_left, df_right, left_on="laci", right_on="laci", how="left").drop("laci_y",
axis=1)
```

7.2.3 concat（拼接）

`concat` 用于沿行或列方向拼接多个 DataFrame：

```
df1 = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
df2 = pd.DataFrame({"A": [5, 6], "B": [7, 8]})

# 按行拼接（纵向）
pd.concat([df1, df2], axis=0)   # 4行2列
#    A  B
# 0  1  3
# 1  2  4
# 0  5  7
# 1  6  8

# 忽略索引
pd.concat([df1, df2], axis=0, ignore_index=True)
#    A  B
# 0  1  3
# 1  2  4
# 2  5  7
# 3  6  8

# 按列拼接（横向）
df3 = pd.DataFrame({"C": [9, 10]})
pd.concat([df1, df3], axis=1)   # 2行3列
#    A  B   C
# 0  1  3   9
# 1  2  4  10
```

> 本项目中，如果数据分多个文件，可以用 concat 按行拼接。但本项目原始数据是一个文件，不需要 concat。

7.2.4 join（索引连接）

`join` 按索引连接，是 merge 的简化版：

```
df1 = pd.DataFrame({"A": [1, 2]}, index=["a", "b"])
df2 = pd.DataFrame({"B": [3, 4]}, index=["a", "b"])

df1.join(df2)   # 按索引连接
#    A  B
# a  1  3
# b  2  4
```

> 本项目中用 merge 即可，join 用得少。

7.2.5 本项目中的合并场景

1. **基站关联补经纬度**：左表（清洗后数据）merge 右表（基站经纬度），左连接；
2. **出行目的匹配**：驻留段中心 merge 目的静态数据，找到最近的目的点；
3. **出行方式匹配**：出行段点 merge 方式静态数据，找到最近的地铁站/公交站；
4. **结果合并**：各模块结果 merge 到主表。

7.3 项目实现步骤

1. 练习 merge 四种连接类型；
2. 练习 concat 拼接；
3. 模拟项目中的基站关联。

7.4 完整代码（合并与连接练习）

```
# ============================================================
# 课时7：合并与连接
# ============================================================

import pandas as pd

# ------------------------------------------------------------
# 1. merge 四种连接类型
# ------------------------------------------------------------
print("=== 1. merge 四种连接类型 ===")

# 左表：用户数据
df_left = pd.DataFrame({
    "imsi": ["A", "B", "C", "D"],
    "laci": ["16789-67567924", "16789-67567925", "16790-67567926", "16791-67567927"]
})

# 右表：基站经纬度
df_right = pd.DataFrame({
    "laci": ["16789-67567924", "16789-67567925", "16790-67567926", "16792-67567928"],
    "lon": [123.41, 123.42, 123.43, 123.44],
    "lat": [41.80, 41.81, 41.82, 41.83]
})

print(f"左表:\n{df_left}")
print(f"\n右表:\n{df_right}")

# 内连接
df_inner = pd.merge(df_left, df_right, on="laci", how="inner")
print(f"\n内连接（只保留匹配的）:\n{df_inner}")

# 左连接
df_left_join = pd.merge(df_left, df_right, on="laci", how="left")
print(f"\n左连接（保留左表所有）:\n{df_left_join}")

# 右连接
```

```
df_right_join = pd.merge(df_left, df_right, on="laci", how="right")
print(f"\n右连接（保留右表所有）:\n{df_right_join}")

# 外连接
df_outer = pd.merge(df_left, df_right, on="laci", how="outer")
print(f"\n外连接（保留两边所有）:\n{df_outer}")

# ------------------------------------------------------------
# 2. 项目模拟：基站关联补经纬度
# ------------------------------------------------------------
print("\n=== 2. 项目模拟：基站关联补经纬度 ===")

# 模拟原始数据（无经纬度）
raw = pd.DataFrame({
    "imsi": ["A", "A", "B", "B", "C"],
    "time": ["0800", "0801", "0800", "0801", "0800"],
    "lac_id": ["16789", "16789", "16790", "16791", "16792"],
    "cell_id": ["67567924", "67567925", "67567926", "67567927", "67567928"]
})

# 基站表
bs = pd.DataFrame({
    "laci": ["16789-67567924", "16789-67567925", "16790-67567926", "16792-67567928"],
    "lon": [123.41, 123.42, 123.43, 123.45],
    "lat": [41.80, 41.81, 41.82, 41.84]
})

# 步骤1：拼接 laci
raw["laci"] = raw["lac_id"] + "-" + raw["cell_id"]
print(f"拼接 laci 后:\n{raw}")

# 步骤2：左连接关联基站
merged = pd.merge(raw, bs, on="laci", how="left")
print(f"\n左连接后:\n{merged}")

# 步骤3：删除未匹配的行（经纬度为空）
cleaned = merged.dropna(subset=["lon", "lat"])
print(f"\n删除未匹配后: {len(cleaned)} 行（原{len(merged)}行）")
```

```
print(cleaned)


# ------------------------------------------------------------
# 3. concat 拼接
# ------------------------------------------------------------
print("\n=== 3. concat 拼接 ===")
df1 = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
df2 = pd.DataFrame({"A": [5, 6], "B": [7, 8]})

# 按行拼接
df_row = pd.concat([df1, df2], axis=0, ignore_index=True)
print(f"按行拼接:\n{df_row}")

# 按列拼接
df3 = pd.DataFrame({"C": [9, 10]})
df_col = pd.concat([df1, df3], axis=1)
print(f"\n按列拼接:\n{df_col}")

print("\n=== 关键总结 ===")
print("1. merge 类似 SQL JOIN，四种连接类型：inner/left/right/outer")
print("2. 本项目基站关联用左连接（how='left'），保留所有原始数据")
print("3. 未匹配到基站的行经纬度为 NaN，后续 dropna 删除")
print("4. concat 用于拼接多个 DataFrame（按行或按列）")
print("5. join 按索引连接，是 merge 的简化版")
```

7.5 运行结果与解读

运行后展示 merge 四种连接、基站关联模拟、concat。特别注意：
- 本项目基站关联用左连接，保留所有原始数据；
- 未匹配到基站的经纬度为 NaN，后续 `dropna(subset=["lon", "lat"])` 删除；
- 这是清洗规则第6条的核心实现。

7.6 当堂练习

1. 创建两个 DataFrame，分别用 inner/left/right/outer 连接，观察结果差异；
2. 模拟基站关联：左表有5条数据，右表只有3条匹配，用左连接后观察 NaN；
3. 思考：为什么本项目用左连接而不是内连接？（提示：左连接保留所有原始数据，可以统计有多少行未匹配到基站；内连接会直接丢弃未匹配的行，无法统计）

7.7 课时小结

- merge：类似 SQL JOIN，四种连接类型 inner/left/right/outer；
- 本项目基站关联用左连接，未匹配的经纬度为 NaN，后续删除；
- concat：按行或按列拼接多个 DataFrame；
- join：按索引连接，merge 的简化版；
- 合并后注意检查 NaN，决定删除或填充。

课时8：项目实战——数据清洗模块完整实现

8.1 课时目标

实现完整的数据清洗模块（9条规则），这是本项目的第一个核心模块。

8.2 知识点深度讲解

8.2.1 9条清洗规则详解

1. **字段抽取**：从原始数据10列中提取 imsi、time、lac_id、cell_id 4列；
2. **去脏 imsi**：imsi 包含 `#`、`*`、`^`、空格等特殊字符的行删除；
3. **去空 lac_id**：lac_id 或 cell_id 为空的行删除；
4. **时间格式转换**：毫秒时间戳转 YYYYMMDDHHMMSS 格式（本项目数据已经是该格式，跳过）；
5. **过滤目标日期**：只保留 2018-10-03 的数据（time 前8位 = 20181003）；
6. **基站关联补经纬度**：拼接 laci = lac_id + "-" + cell_id，左连接基站表；
7. **按 imsi+time 排序**：每个用户按时间升序排列；
8. **分钟去重**：同一用户同一分钟（time 前12位）只保留第一条；
9. **过滤低频用户**：单用户数据量 < 20 条的用户删除。

8.2.2 清洗流程设计

```
原始数据（53万行，10列）
  ↓ 规则1：字段抽取
（53万行，4列：imsi/time/lac_id/cell_id）
  ↓ 规则2：去脏imsi
（约52万行）
  ↓ 规则3：去空lac_id/cell_id
（约52万行）
  ↓ 规则4：时间格式转换（本项目跳过）
  ↓ 规则5：过滤目标日期20181003
（约52万行）
  ↓ 规则6：基站关联补经纬度
（约52万行，新增lon/lat列）
  ↓ 删除经纬度为空的行
（约50万行）
  ↓ 规则7：按imsi+time排序
  ↓ 规则8：分钟去重
（约12万行，抽样2万行后约1.2万行）
  ↓ 规则9：过滤低频用户（<20条）
（约186用户，1.2万行）
  ↓
清洗后数据（输出 cleaned_data.csv）
```

> 注意：实际运行时为了快速验证，会抽样2万行。全量53万行也能跑，但耗时稍长。

8.2.3 关键实现细节

1. **读取时指定 dtype**：imsi/time/lac_id/cell_id 全部按字符串读取；
2. **去脏 imsi**：用正则 `imsi.str.contains(r"[#* ^\s]")` 检测；
3. **基站关联**：拼接 laci 后左连接，注意基站表的 laci 也是字符串；
4. **分钟去重**：提取 time 前12位（YYYYMMDDHHMM），按 imsi+分钟去重；
5. **过滤低频用户**：`groupby("imsi").filter(lambda x: len(x) >= 20)`；
6. **输出**：`to_csv(index=False, encoding="utf-8")`。

8.3 项目实现步骤

1. 读取原始数据和基站数据（指定 dtype）；

2. 依次执行9条清洗规则；

3. 输出清洗后数据；

4. 验证清洗结果（行数、用户数、经纬度范围）。

8.4 完整代码（data_clean.py）

```
# ============================================================
# 项目实战：数据清洗模块 data_clean.py
# 功能：执行9条清洗规则，输出清洗后数据
# 运行：python data_clean.py
# ============================================================

import os
import pandas as pd
import numpy as np

# ------------------------------------------------------------
# 配置
# ------------------------------------------------------------
# 数据根目录
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统(\09.交通时空大数据分析系统\B-项目数据"
# 输出目录
OUTPUT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# 目标日期
TARGET_DATE = "20181003"
# 低频用户阈值（单用户最少数据量）
MIN_RECORDS_PER_USER = 20
# 抽样行数（None表示全量，20000表示抽样2万行快速验证）
SAMPLE_ROWS = 20000


def clean_data():
    """执行9条清洗规则，返回清洗后的DataFrame。"""

    # ============================================================
    # 规则0：读取数据
    # ============================================================
    print("=" * 60)
```

```
print("规则0：读取数据")
print("=" * 60)

raw_path = os.path.join(DATA_DIR, "原始数据.csv")
bs_path = os.path.join(DATA_DIR, "基站经纬度数据.csv")

# 读取原始数据（关键：指定dtype=str）
print(f"读取原始数据：{raw_path}")
if SAMPLE_ROWS:
    df = pd.read_csv(raw_path, nrows=SAMPLE_ROWS, dtype={
        "imsi": str, "time": str, "lac_id": str, "cell_id": str
    })
    print(f"  抽样 {SAMPLE_ROWS} 行")
else:
    df = pd.read_csv(raw_path, dtype={
        "imsi": str, "time": str, "lac_id": str, "cell_id": str
    })
print(f"  原始数据：{len(df)} 行，列：{df.columns.tolist()}")

# 读取基站数据
print(f"读取基站数据：{bs_path}")
df_bs = pd.read_csv(bs_path, dtype={"laci": str})
print(f"  基站数据：{len(df_bs)} 行")

# ============================================================
# 规则1：字段抽取
# ============================================================
print("\n" + "=" * 60)
print("规则1：字段抽取（imsi/time/lac_id/cell_id）")
print("=" * 60)
df = df[["imsi", "time", "lac_id", "cell_id"]].copy()
print(f"  抽取后：{len(df)} 行，列：{df.columns.tolist()}")

# ============================================================
# 规则2：去脏imsi（含#*^空格等特殊字符）
# ============================================================
print("\n" + "=" * 60)
print("规则2：去脏imsi")
```

```
print("=" * 60)
before = len(df)
# 检测包含特殊字符的imsi
dirty_mask = df["imsi"].str.contains(r"[#* ^\s]", na=False)
print(f"  脏imsi行数: {dirty_mask.sum()}")
df = df[~dirty_mask].copy()
print(f"  去脏后: {len(df)} 行（删除 {before - len(df)} 行）")

# ============================================================
# 规则3：去空lac_id/cell_id
# ============================================================
print("\n" + "=" * 60)
print("规则3：去空lac_id/cell_id")
print("=" * 60)
before = len(df)
# lac_id或cell_id为空（空字符串或NaN）
df["lac_id"] = df["lac_id"].fillna("").str.strip()
df["cell_id"] = df["cell_id"].fillna("").str.strip()
empty_mask = (df["lac_id"] == "") | (df["cell_id"] == "")
print(f"  空lac_id/cell_id行数: {empty_mask.sum()}")
df = df[~empty_mask].copy()
print(f"  去空后: {len(df)} 行（删除 {before - len(df)} 行）")

# ============================================================
# 规则4：时间格式转换（本项目数据已是YYYYMMDDHHMMSS，跳过）
# ============================================================
print("\n" + "=" * 60)
print("规则4：时间格式转换（本项目数据已是目标格式，跳过）")
print("=" * 60)
print(f"  time 样例: {df['time'].iloc[0]}")

# ============================================================
# 规则5：过滤目标日期
# ============================================================
print("\n" + "=" * 60)
print(f"规则5：过滤目标日期 {TARGET_DATE}")
print("=" * 60)
before = len(df)
```

```
# time前8位 = 日期
df["date"] = df["time"].str[:8]
target_mask = df["date"] == TARGET_DATE
print(f"  目标日期行数: {target_mask.sum()}")
df = df[target_mask].copy()
print(f"  过滤后: {len(df)} 行（删除 {before - len(df)} 行）")

# ============================================================
# 规则6：基站关联补经纬度
# ============================================================
print("\n" + "=" * 60)
print("规则6：基站关联补经纬度")
print("=" * 60)
before = len(df)
# 拼接 laci = lac_id + "-" + cell_id
df["laci"] = df["lac_id"] + "-" + df["cell_id"]
print(f"  laci 样例: {df['laci'].iloc[0]}")

# 左连接基站表
df = pd.merge(df, df_bs, on="laci", how="left")
print(f"  关联后: {len(df)} 行")

# 删除经纬度为空的行（未匹配到基站）
no_geo = df["lon"].isna() | df["lat"].isna()
print(f"  未匹配到基站的行数: {no_geo.sum()}")
df = df.dropna(subset=["lon", "lat"]).copy()
print(f"  删除未匹配后: {len(df)} 行（删除 {before - len(df)} 行）")

# ============================================================
# 规则7：按imsi+time排序
# ============================================================
print("\n" + "=" * 60)
print("规则7：按imsi+time排序")
print("=" * 60)
df = df.sort_values(by=["imsi", "time"]).reset_index(drop=True)
print(f"  排序完成，用户数: {df['imsi'].nunique()}")

# ============================================================
```

```
# 规则8：分钟去重
# ============================================================
print("\n" + "=" * 60)
print("规则8：分钟去重（同一用户同一分钟只保留一条）")
print("=" * 60)
before = len(df)
# 提取分钟（time前12位 = YYYYMMDDHHMM）
df["minute"] = df["time"].str[:12]
# 按imsi+minute去重，保留第一条
df = df.drop_duplicates(subset=["imsi", "minute"], keep="first").copy()
print(f"  去重后：{len(df)} 行（删除 {before - len(df)} 行）")

# ============================================================
# 规则9：过滤低频用户
# ============================================================
print("\n" + "=" * 60)
print(f"规则9：过滤低频用户（< {MIN_RECORDS_PER_USER} 条）")
print("=" * 60)
before_users = df["imsi"].nunique()
before_rows = len(df)
# 方法1：用filter
df = df.groupby("imsi").filter(lambda x: len(x) >= MIN_RECORDS_PER_USER).copy()
after_users = df["imsi"].nunique()
after_rows = len(df)
print(f"  过滤前：{before_users} 用户，{before_rows} 行")
print(f"  过滤后：{after_users} 用户，{after_rows} 行")
print(f"  删除：{before_users - after_users} 用户，{before_rows - after_rows} 行")

# ============================================================
# 输出
# ============================================================
print("\n" + "=" * 60)
print("输出清洗后数据")
print("=" * 60)

# 选择输出列
output_cols = ["imsi", "time", "lac_id", "cell_id", "lon", "lat"]
df_output = df[output_cols].copy()
```

```
    output_path = os.path.join(OUTPUT_DIR, "cleaned_data.csv")
    df_output.to_csv(output_path, index=False, encoding="utf-8")
    print(f"  输出文件: {output_path}")
    print(f"  输出行数: {len(df_output)}")
    print(f"  输出列: {output_cols}")

    # ============================================================
    # 验证
    # ============================================================
    print("\n" + "=" * 60)
    print("清洗结果验证")
    print("=" * 60)
    print(f"  总行数: {len(df_output)}")
    print(f"  用户数: {df_output['imsi'].nunique()}")
    print(f"  日期范围: {df_output['time'].str[:8].unique()}")
    print(f"  经度范围: [{df_output['lon'].min():.4f}, {df_output['lon'].max():.4f}]")
    print(f"  纬度范围: [{df_output['lat'].min():.4f}, {df_output['lat'].max():.4f}]")
    print(f"  缺失值: lon={df_output['lon'].isna().sum()}, lat=
{df_output['lat'].isna().sum()}")
    print(f"  每用户平均数据量: {len(df_output) / df_output['imsi'].nunique():.1f} 条")

    return df_output


if __name__ == "__main__":
    clean_data()
    print("\n" + "=" * 60)
    print("数据清洗完成！")
    print("=" * 60)
```

8.5 运行结果与解读

运行后（用 `D:\soft\python\python.exe data_clean.py`）应看到：
- 抽样2万行 → 清洗后约1.2万行、186用户；
- 每一步都打印删除行数，便于追踪数据量变化；
- 验证部分打印总行数、用户数、经纬度范围、缺失值。

> 关键验证点：
> 1. 经纬度无缺失（规则6删除了未匹配的行）；
> 2. 日期只有20181003（规则5过滤）；
> 3. 每用户数据量 >= 20（规则9过滤）；
> 4. 经纬度在沈阳范围内（123.3-123.5, 41.7-41.9）。

8.6 当堂练习

1. 修改 `SAMPLE_ROWS = None`（全量），运行清洗，观察全量数据的清洗结果；
2. 修改 `MIN_RECORDS_PER_USER = 50`，观察过滤后用户数变化；
3. 在规则2中增加对 imsi 长度的校验（正常imsi为15-20位），删除长度异常的imsi；
4. 思考：为什么分钟去重要在排序之后？（提示：去重保留第一条，排序后第一条是时间最早的）

8.7 课时小结

- 9条清洗规则全部实现，每步打印数据量变化；
- 关键坑：dtype=str 读取、左连接基站、分钟去重、过滤低频用户；
- 输出 cleaned_data.csv，供后续模块使用；
- 验证：无缺失值、日期正确、用户数据量达标、经纬度范围合理。

三、本日小结

1. **Pandas设计思想**：DataFrame 二维表格，底层基于 NumPy，支持异构类型；
2. **数据读取与写入**：read_csv/to_csv，关键参数 encoding/dtype，三个坑；
3. **数据选择与过滤**：loc/iloc、布尔索引、query、字符串过滤；
4. **数据清洗**：缺失值、重复值、异常值，本项目9条清洗规则；
5. **数据转换与类型处理**：astype、.str操作、.dt时间处理、列操作；
6. **分组聚合与透视表**：groupby、agg、filter、transform、pivot_table；
7. **合并与连接**：merge四种连接、concat、join；
8. **项目实战**：完整实现 data_clean.py，9条规则全部落地。

四、本日作业

1. 运行 data_clean.py，记录每一步的数据量变化，分析哪一步删除最多；

2. 用 groupby 统计清洗后每个用户的数据量分布，画直方图；

3. 思考：如果原始数据的 time 是毫秒时间戳（如1538524800000），规则4应该怎么实现？

4. 预习 Day04：思考"出行和驻留识别的三阶段算法分别解决什么问题"。